Python: Threading, NumPy and pandas

From basics to optimisation, with measured numbers

- Part 1 — Threading and the GIL
  - 1.1 What the GIL is
  - 1.2 The measurements
  - 1.3 The rule
  - 1.4 When the GIL is released
  - 1.5 `threading` basics
  - 1.6 Does the GIL make Python thread-safe?
  - 1.7 `concurrent.futures` — the modern interface
  - 1.8 `multiprocessing`
  - 1.9 `asyncio`
  - 1.10 The free-threaded build
  - 1.11 Interview answers
- Part 2 — NumPy
  - 2.1 What an ndarray actually is
  - 2.2 Why it's fast
  - 2.3 Creation and dtypes
  - 2.4 Views versus copies — the thing to get right
  - 2.5 Broadcasting
  - 2.6 Optimisation technique 1: eliminate temporaries
  - 2.7 Optimisation technique 2: memory layout and iteration order
  - 2.8 Optimisation technique 3: pick the right function
  - 2.9 Optimisation technique 4: stride tricks
  - 2.10 Optimisation technique 5: avoid `object` dtype
  - 2.11 Structured arrays
  - 2.12 The optimisation checklist
- Part 3 — pandas
  - 3.1 What it is
  - 3.2 Selection — `loc` vs `iloc`
  - 3.3 The performance table
  - 3.4 groupby
  - 3.5 The `category` dtype — a large, easy win
  - 3.6 Time series
  - 3.7 Joins
  - 3.8 `SettingWithCopyWarning`
  - 3.9 Memory reduction
  - 3.10 When to leave pandas
- Part 4 — Interview Questions
- The three numbers worth remembering

Part 1 — Threading and the GIL

1.1 What the GIL is

The **Global Interpreter Lock** is a single mutex protecting the CPython interpreter. **Only one thread executes Python bytecode at a time.**

Why it exists: CPython uses reference counting for memory management. Every object holds a count of how many references point at it, updated constantly. Without the GIL, every one of those increments and decrements would need to

be atomic — which would slow single-threaded code down substantially. The GIL is the trade: simple, fast single-threaded execution, at the cost of no parallel bytecode.

What it is not: it isn't a design mistake nobody noticed. It's a deliberate tradeoff that's been debated for 30 years. Several attempts to remove it made single-threaded code 30–40% slower.

1.2 The measurements

CPU-bound pure Python, same total work:

	Time	Speedup
1 thread	362 ms	1.00×
2 threads	347 ms	**1.04×**
4 threads	334 ms	**1.09×**

Essentially nothing. Four threads gave 9%, and that's mostly noise. The GIL serialises them.

I/O-bound (8 × 100 ms sleeps):

	Time	Speedup
Serial	801 ms	1.00×
8 threads	102 ms	**7.8×**

Nearly linear. Because `time.sleep`, file I/O and network calls **release the GIL** while waiting.

1.3 The rule

Workload	Threads help?	Use
CPU-bound pure Python	**No**	`multiprocessing`, or C extension
I/O-bound (network, disk)	**Yes**	`threading` or `asyncio`
NumPy / C extensions	**Yes, if the extension releases it**	`threading`
Mixed		threads for I/O, processes for compute

1.4 When the GIL is released

Any C code can release it around work that doesn't touch Python objects:

- `time.sleep`
- File and socket I/O
- Most NumPy operations on large arrays
- `hashlib`, compression, most C extensions
- Your own pybind11 code, if you release it explicitly

```
{
    py::gil_scoped_release release;    // RAII: reacquired at the closing brace
    heavy_cpp_work();                  // must touch NO Python objects
}
```

Safe only because nothing inside touches a Python object. Touching one after release is a **crash**, not a slowdown.

This is why a batch binding that converts to `std::vector` first can release the GIL, while a list-of-tuples binding cannot — the latter unpacks Python objects inside the loop.

1.5 `threading` basics

```
import threading

def worker(name, results, idx):
    results[idx] = expensive(name)

threads = []
results = [None] * 4
for i, name in enumerate(names):
    t = threading.Thread(target=worker, args=(name, results, i))
    t.start()
    threads.append(t)
for t in threads:
    t.join()              # wait for completion
```

Daemon threads die when the main thread exits: `t = Thread(target=f, daemon=True)`. Use for background work that shouldn't block shutdown.

Synchronisation primitives

```
lock = threading.Lock()            # basic mutex
with lock:                         # ALWAYS use the context manager
    shared_state += 1

rlock = threading.RLock()          # re-entrant: same thread can acquire twice
sem = threading.Semaphore(5)       # allow at most 5 concurrent
event = threading.Event()          # signal between threads
event.set(); event.wait()
cond = threading.Condition()       # wait for a predicate
barrier = threading.Barrier(3)     # all 3 must arrive before any proceeds
```

`queue.Queue` is the thread-safe way to pass data — it handles the locking for you:

```
from queue import Queue
q = Queue(maxsize=100)
q.put(item)                        # blocks when full
item = q.get()                     # blocks when empty
q.task_done()                      # mark complete
q.join()                           # wait for all tasks
```

Prefer a queue over shared mutable state plus a lock. It's harder to get wrong.

1.6 Does the GIL make Python thread-safe?

No — and this is the question that catches people out.

The GIL guarantees one thread runs bytecode at a time. It does **not** make multi-step operations atomic:

```
counter += 1      # THREE bytecodes: LOAD, ADD, STORE
```

A thread switch can happen between them, so two threads can both load the same value and both store the same result — one increment lost. Exactly the same bug as `x++` in C++.

What is atomic (single bytecode, in CPython): `list.append`, `dict[k] = v`, `x = y`. **What isn't:** `+=`, `if k not in d: d[k] = v`, anything read-modify-write.

Don't rely on which is which. It's a CPython implementation detail, not a language guarantee. Use a lock.

1.7 `concurrent.futures` — the modern interface

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

with ThreadPoolExecutor(max_workers=8) as ex:       # I/O-bound
    results = list(ex.map(fetch_url, urls))

with ProcessPoolExecutor(max_workers=4) as ex:      # CPU-bound
    results = list(ex.map(heavy_compute, chunks))
```

Same interface, swap the executor. Also:

```
futures = [ex.submit(f, x) for x in items]
for fut in as_completed(futures):        # process as they finish
    result = fut.result()                # re-raises any exception
```

Exceptions surface at `.result()`, not at submit — a common source of silently swallowed errors.

1.8 `multiprocessing`

Separate processes, so separate GILs and genuine parallelism.

```
from multiprocessing import Pool
with Pool(4) as p:
    results = p.map(heavy, items)
```

The costs, which are real:

- **Pickling.** Arguments and results are serialised. Large DataFrames are expensive to send.
- **Startup.** `fork` on Linux is cheap; `spawn` (Windows, macOS default) re-imports your module.
- **No shared memory by default.** Use `multiprocessing.shared_memory` or `Array` if you need it.
- **Debugging is harder.** Exceptions in children are less visible.

Rule of thumb: worth it when the work per item substantially exceeds the pickling cost. For small items you'll go slower.

1.9 `asyncio`

Single-threaded cooperative concurrency. No GIL contention because there's only one thread.

```
import asyncio

async def fetch(session, url):
    async with session.get(url) as r:
        return await r.text()

async def main():
    async with aiohttp.ClientSession() as s:
        results = await asyncio.gather(*[fetch(s, u) for u in urls])

asyncio.run(main())
```

When to use it over threads: thousands of concurrent I/O operations. Threads cost ~8 MB of stack each; coroutines cost a few KB.

The trap: one blocking call blocks *everything*, because there's only one thread. Never call synchronous I/O or CPU-heavy code inside a coroutine — use `run_in_executor` for that.

1.10 The free-threaded build

Python 3.13 shipped an **experimental** build with the GIL removed (PEP 703), enabled with `--disable-gil`. Real

parallelism for pure Python threads.

Caveats: single-threaded code is somewhat slower, and C extensions need updating to be safe. Not yet the default. Worth knowing it exists.

1.11 Interview answers

“Why doesn’t threading speed up my CPU-bound Python?” The GIL — only one thread executes bytecode at a time. Measured: four threads on the same CPU-bound work gave 1.09×, essentially nothing. Use multiprocessing, or push the hot part into C where the GIL can be released.

“So threads are useless?” No — for I/O they’re excellent, because the GIL is released during the wait. Eight concurrent sleeps went from 801 ms serial to 102 ms threaded, a 7.8× speedup.

“Does the GIL make my code thread-safe?” No. It serialises bytecode, but `counter += 1` is three bytecodes and can be interrupted between them. You still need locks for anything read-modify-write.

“When is it safe to release the GIL from C++?” When the code holding it touches no Python objects. Convert to C++ types first, then release. Touching a Python object without the GIL is a crash, not a slowdown.

“threads, processes or asyncio?” I/O with a handful of tasks → threads. I/O with thousands → asyncio. CPU-bound → processes, or better, move the work into NumPy or C++.

Part 2 — NumPy

2.1 What an ndarray actually is

Three things: a **contiguous block of memory**, a **dtype** (how to interpret each element), and a **shape + strides** (how to navigate it).

```
a = np.arange(12).reshape(3,4)
a.shape       # (3, 4)
a.strides     # (32, 8)  -- bytes to step: 32 to the next row, 8 to the next column
a.dtype       # int64
a.nbytes      # 96
a.flags       # C_CONTIGUOUS, F_CONTIGUOUS, OWNDATA, WRITEABLE
```

Strides are the key idea. A transpose doesn’t move data — it swaps the strides. That’s why `a.T` is instant regardless of size, and why the result is no longer C-contiguous.

2.2 Why it’s fast

Two reasons, and both matter:

1. **No per-element interpreter overhead.** One C loop instead of a million bytecode dispatches.
2. **Contiguous memory** — the CPU prefetcher works, and operations vectorise to SIMD.

Measured, 1M elements:

	Time
Python list comprehension	64.6 ms
`np.array(list)` then vectorise	53.1 ms
NumPy on existing arrays	**1.35 ms**

48× faster. And note the middle row — converting lists to arrays costs almost as much as the Python loop. **The win comes from keeping data in arrays**, not from converting at the last moment.

2.3 Creation and dtypes

```
np.zeros(n); np.ones(n); np.empty(n)      # empty is uninitialised -- fastest
np.full(n, 3.14)
np.arange(0, 10, 0.5)                     # like range, with floats
np.linspace(0, 1, 100)                    # 100 points inclusive
np.random.default_rng(42).random(n)       # modern RNG interface

np.zeros(n, dtype=np.float32)
a.astype(np.int32)                        # makes a COPY
```

dtype	Bytes	Notes
`float64`	8	default; ~15 significant digits
`float32`	4	~7 digits; **half the memory and ~2× faster**
`int64` / `int32`	8 / 4	
`bool`	1	
`object`	8 (pointer)	**falls back to Python speed — avoid**

Measured: `float32` multiply took 0.50 ms vs 1.01 ms for `float64` — **2×**, and half the memory. Worth it when 7 digits suffice, which is most signal work but not money.

Watch for silent upcasting:

```
a32 = np.ones(5, np.float32)
(a32 * 2.0).dtype        # float64! the Python float promoted it
(a32 * np.float32(2)).dtype   # float32
```

2.4 Views versus copies — the thing to get right

```
a = np.arange(10)
b = a[2:5]           # VIEW -- shares memory
b[0] = 99            # a is now modified!

c = a[[2,3,4]]       # COPY (fancy indexing)
d = a[a > 5]         # COPY (boolean mask)
e = a.reshape(2,5)   # VIEW if possible
f = a.T              # VIEW always
g = a.copy()         # explicit copy
```

Check with:

```
b.base is a                 # True for a view
np.shares_memory(a, b)
b.flags['OWNDATA']          # False for a view
```

Measured cost on a 1M array:

	Time
Slice `a[::2]` (view)	**0.0003 ms**
Fancy `a[idx]` (copy)	0.66 ms
Boolean `a[mask]` (copy)	5.66 ms

A view is essentially free; a boolean mask is 19,000× more expensive because it allocates and copies.

Why it matters beyond speed: an accidental view means one calculation silently corrupts another's input. An accidental copy means your in-place update doesn't stick.

2.5 Broadcasting

Shapes align **from the right**; dimensions of size 1 stretch.

```
a = np.ones((3,4))
b = np.ones(4)              # (4,) -> (1,4) -> (3,4)     works
c = np.ones(3)              # (3,) doesn't align         ERROR
a + c[:, np.newaxis]        # (3,1) broadcasts           works
```

Common idiom — normalise each row:

```
normalised = data / data.sum(axis=1, keepdims=True)
```

`keepdims=True` preserves the dimension so broadcasting works without a manual reshape.

The hidden cost: broadcasting doesn't copy, but the *result* is materialised. `a[:, None] * b[None, :]` on two 10,000-element arrays produces a 100M-element array — 800 MB.

2.6 Optimisation technique 1: eliminate temporaries

Every intermediate expression allocates:

```
c = a*2 + b*3      # allocates: (a*2), (b*3), and the sum
```

Use the `out=` parameter and in-place operators:

```
out = np.empty_like(a)
np.multiply(a, 2, out=out)
out += b*3
```

Measured on 1M elements:

	Time
`a*2 + b*3`	2.06 ms
`out=` parameter	1.83 ms
Preallocated + `out=`	**1.18 ms**

~1.7×, and more importantly it's allocation-free — so it's predictable, which matters in a latency-sensitive path.

In-place operators mutate, so know what else references the array:

```
a += b      # in-place, no allocation
a = a + b   # allocates a new array
```

2.7 Optimisation technique 2: memory layout and iteration order

Measured, summing rows vs columns of a 3000×3000 array:

	Time
Row by row	**0.79 ms**
Column by column	**8.70 ms**

11×, same arithmetic. Rows are contiguous in C-order; columns stride by 24,000 bytes, so every element is a separate cache line.

But be careful with the naive version of this claim. I also measured:

```
m.sum(axis=1)   3.55 ms   (C-contiguous)
mt.sum(axis=1)  2.86 ms   (transposed, NOT contiguous)
```

The transpose was *faster*. NumPy's reduction machinery is smart enough to iterate whichever axis it can do contiguously, so the naive "transposed is slower" story doesn't hold for built-in reductions.

The honest rule: iteration order matters enormously when *you* control the loop. NumPy's own reductions often optimise it away. Measure rather than assume.

```
np.ascontiguousarray(a)      # force C-order (copies)
np.asfortranarray(a)         # force column-major
a.flags['C_CONTIGUOUS']
```

2.8 Optimisation technique 3: pick the right function

Measured on 1M elements:

Operation	Time
Python `sum(list)`	6.28 ms
`np.sum(a)`	**0.39 ms**
`a @ b` (dot product)	0.57 ms

And for conditional work:

	Time
`np.where(a>0.5, a, 0)`	5.04 ms
`a * (a>0.5)`	1.43 ms
`np.clip(a, 0.5, None)`	**0.61 ms**

8× between the obvious choice and the right one. `np.where` allocates a boolean array *and* does a select; `clip` is a single pass.

Others worth knowing:

```
np.einsum('ij,jk->ik', a, b)     # explicit contraction, often avoids temporaries
np.dot / @                        # BLAS-backed, uses all cores
np.add.reduce / np.add.accumulate
np.searchsorted(sorted_arr, vals) # binary search, vectorised
np.select([c1,c2],[v1,v2],default) # multi-way where
```

2.9 Optimisation technique 4: stride tricks

Rolling windows without copying. Measured on 200k elements, window 50:

	Time
Python loop over slices	421.80 ms
`sliding_window_view`	5.75 ms
`cumsum` trick	**1.22 ms**

346× for the stride version, 346× again for the cumsum version over the loop.

```
from numpy.lib.stride_tricks import sliding_window_view
windows = sliding_window_view(a, 50)      # shape (n-49, 50), NO COPY
rolling_mean = windows.mean(axis=1)

# but for a mean specifically, cumsum is better still:
cs = np.cumsum(np.insert(a, 0, 0))
rolling_mean = (cs[50:] - cs[:-50]) / 50
```

The lesson: the stride trick avoids the copy, but the *algorithm* change (prefix sums) beats it. Reach for the better

algorithm before the cleverer memory trick.

Caution on cumsum: it accumulates floating-point error over long arrays. For 1M elements of similar magnitude it's fine; for values spanning many orders of magnitude, use the windowed version.

2.10 Optimisation technique 5: avoid `object` dtype

```
a = np.array([1, 'two', 3.0])     # dtype=object -- Python speed, no vectorisation
```

Any `object` array falls back to per-element Python calls. Check `a.dtype` if something is mysteriously slow.

2.11 Structured arrays

```
dt = np.dtype([("symbol","S16"), ("qty","i8"), ("price","f8")])
fills = np.zeros(1000, dtype=dt)
fills["price"] *= 1.01           # vectorised on one field
```

Two traps I hit for real:

1. **A field view is STRIDED, not contiguous.** `arr["symbol"]` has a stride equal to the **full record size**, not the field's itemsize. Passing `.data` to C++ and walking it with `base + i*itemsize` reads interleaved bytes from neighbouring fields — it produced *plausible-looking garbage* that surfaced three tests downstream.
2. `'S'` **fields are NUL-padded, not NUL-terminated.** A string that exactly fills the field has no terminator, so bound your search by itemsize.

Alternative: for pure numeric data, separate arrays (SoA) are usually faster and simpler than a structured array.

2.12 The optimisation checklist

1. **Vectorise** — remove Python loops. 48× measured.
2. **Keep data in arrays** — converting lists costs as much as the loop did.
3. **Right function** — `clip` over `where`, 8×.
4. **Better algorithm** — cumsum over sliding windows, 5×.
5. **Eliminate temporaries** — `out=` and in-place, 1.7× and allocation-free.
6. **Right dtype** — `float32` where precision allows, 2×.
7. **Watch views vs copies** — a boolean mask copy is 19,000× a slice.
8. **Iteration order** — 11× for row vs column, when you control the loop.
9. **Then** consider Numba, Cython, or C++.

Part 3 — pandas

3.1 What it is

A `DataFrame` is a dict of `Series`, each backed by a NumPy array, plus an **Index**.

The index is the part people underuse. It's not just row numbers — it's a lookup structure that makes alignment, joins and time-series operations fast.

```
df.index; df.columns; df.dtypes; df.shape
df.info(memory_usage='deep')      # dtypes AND real memory
df.describe()
```

3.2 Selection — `loc` vs `iloc`

```
df.loc['2024-01-15']              # by LABEL
df.loc['2024-01-15':'2024-01-20'] # label slice is INCLUSIVE of the end
df.iloc[0]                        # by POSITION
df.iloc[0:5]                      # position slice is EXCLUSIVE (like Python)
df.at['x','col']                  # scalar access -- faster than .loc
df.iat[0,1]                       # scalar by position
```

The inclusive/exclusive difference is a real source of off-by-one bugs.

```
df[df.qty > 100]                          # boolean mask
df[(df.qty > 100) & (df.sym == 'TCS')]    # & and |, NOT and/or, and MIND THE PARENTHESES
df.query('qty > 100 and sym == "TCS"')    # sometimes cleaner
```

Python's `and` / `or` don't work elementwise — they call `__bool__` on the whole Series and raise.

3.3 The performance table

Measured, 200,000 rows, computing `qty * price`:

Method	Time	vs vectorised
`iterrows`	~4,305 ms	**6,000× slower**
`apply(axis=1)`	~1,906 ms	2,700×
`itertuples`	127 ms	180×
Vectorised	**0.7 ms**	1×
`.values` vectorised	**0.5 ms**	0.7×

`iterrows` constructs a Series object per row. That's why it's catastrophic — 200,000 object allocations plus dtype coercion.

If you must iterate, use `itertuples` — 34× faster than `iterrows` because it yields lightweight namedtuples.

And `.values` beats the Series operation by dropping to raw NumPy, skipping index alignment.

3.4 groupby

```
df.groupby('sym')['qty'].sum()
df.groupby('sym').agg({'qty':'sum', 'price':'mean'})
df.groupby(['sym','date'])['qty'].sum()
df.groupby('sym')['qty'].transform('mean')     # broadcasts back to original shape
df.groupby('sym').filter(lambda g: len(g) > 100)
```

Measured on 200k rows:

	Time
`groupby.sum()`	12.0 ms
`groupby.agg({...})`	13.6 ms
`groupby.apply(lambda)`	16.2 ms

Built-in aggregations are Cython-optimised. `apply` with a lambda falls back to Python per group — tolerable with 4 groups, terrible with 10,000.

3.5 The `category` dtype — a large, easy win

```
df['sym'] = df['sym'].astype('category')
```

Measured on 200k rows with 4 distinct symbols:

	Memory	groupby
`object` dtype	14.0 MB	11.9 ms
`category` dtype	**3.4 MB**	**2.8 ms**

4× less memory and 4× faster groupby. For any column with few distinct values relative to row count — symbols, exchanges, order types — this is close to free.

3.6 Time series

```
df.index = pd.to_datetime(df.date)
df.resample('1min').agg({'price':'ohlc', 'qty':'sum'})
df.rolling(20).mean()
df.rolling('5min').mean()              # time-based window
df.expanding().sum()
df.ewm(span=20).mean()                 # exponentially weighted
df.shift(1)                            # lag
df.pct_change()
df.between_time('09:15','15:30')
df.tz_localize('UTC').tz_convert('Asia/Kolkata')
```

`rolling` and `resample` are the two that make pandas worth using for market data at all.

`shift` is how you avoid look-ahead bias: compare today's signal against `df.close.shift(-1)` for tomorrow's return, never today's.

3.7 Joins

```
pd.merge(a, b, on='sym', how='inner')          # inner/left/right/outer
pd.merge(a, b, left_on='x', right_on='y')
a.join(b)                                       # on index -- faster
pd.concat([a,b])                                # stack rows
pd.merge_asof(trades, quotes, on='time')        # NEAREST PRIOR match
```

`merge_asof` is the market-data join. Matching each trade to the prevailing quote is exactly what it does, and doing it by hand is both slow and easy to get wrong.

Joining on the index is faster than on a column — it's already a hash or sorted structure.

3.8 `SettingWithCopyWarning`

```
subset = df[df.qty > 0]     # may be a view or a copy -- pandas can't tell you which
subset['x'] = 1             # WARNING: the write may not reach df
```

Fix by being explicit:

```
df.loc[df.qty > 0, 'x'] = 1        # modify the original in one step
subset = df[df.qty > 0].copy()     # or take an explicit copy
```

Same underlying issue as NumPy views — you don't know whether you're aliasing.

3.9 Memory reduction

```
df.memory_usage(deep=True)                  # deep=True counts object strings properly
df['x'] = pd.to_numeric(df.x, downcast='integer')   # int64 -> int8 if it fits
df['sym'] = df['sym'].astype('category')
df['price'] = df['price'].astype(np.float32)
pd.read_csv(f, usecols=[...], dtype={...})   # don't load what you won't use
pd.read_csv(f, chunksize=100_000)            # process in chunks
```

Prefer Parquet over CSV for anything you'll re-read: columnar, typed, compressed, and typically 10× faster to load.

3.10 When to leave pandas

pandas has real per-operation overhead — index alignment, dtype checking, block management. For a tight numeric loop, drop to NumPy:

```
prices = df.price.values        # raw ndarray, no index
result = compute(prices)        # pure NumPy
df['result'] = result
```

Measured above: `.values` was 1.4× faster than the Series operation, and that gap widens for chained operations.

Alternatives worth knowing by name: Polars (Rust, lazy, multi-threaded), DuckDB (SQL over DataFrames), Dask (out-of-core), Arrow (the underlying columnar format).

Part 4 — Interview Questions

"Why doesn't threading speed up CPU-bound Python?" The GIL — one thread executes bytecode at a time. Four threads on the same work measured 1.09×. For I/O it's the opposite: eight concurrent sleeps went 801 ms → 102 ms, 7.8×, because I/O releases the GIL.

"Does the GIL make Python thread-safe?" No. `counter += 1` is three bytecodes and a switch can happen between them. You still need locks for read-modify-write.

"threads, processes, or asyncio?" I/O with a few tasks → threads. I/O with thousands → asyncio, since threads cost ~8 MB of stack each. CPU-bound → processes, or better, move the work into NumPy or C++.

"How would you speed up slow Python?" Profile first — `cProfile`, then `line_profiler`. Then: vectorise with NumPy (48× measured), pick the right function (`clip` over `where` was 8×), improve the algorithm (cumsum over sliding windows, 5×), eliminate temporaries with `out=`. Only then reach for Numba or C++.

"View or copy in NumPy?" Slicing gives a view, fancy indexing and boolean masks give copies. A slice measured 0.0003 ms against 5.7 ms for a boolean mask on 1M elements. Check with `np.shares_memory`. Getting it wrong means either silent aliasing or unexpected memory.

"Why is `iterrows` bad?" It constructs a Series object per row. Measured 6,000× slower than the vectorised form on 200k rows. If you truly must iterate, `itertuples` is 34× faster than `iterrows`.

"How do you reduce DataFrame memory?" `category` dtype for low-cardinality columns — measured 4× less memory and 4× faster groupby. Downcast numerics, use `float32` where precision allows, load only the columns you need, and use Parquet rather than CSV.

"What is broadcasting?" Shapes align from the right and size-1 dimensions stretch, so operations on different-shaped arrays work without explicit loops. It doesn't copy the inputs, but the result is materialised — so `a[:,None] * b[None,:]` on two 10k arrays produces 800 MB.

"When would you leave pandas for NumPy?" When per-operation overhead dominates — index alignment and dtype checks cost more than the arithmetic in a tight loop. `.values` gives you the raw array.

"How do you avoid look-ahead bias in a backtest?" `shift`. Compare today's signal against `close.shift(-1)` for tomorrow's return. Any calculation that uses same-bar data to predict the same bar is leaking.

The three numbers worth remembering

48×	vectorising a Python loop in NumPy
6,000×	vectorised pandas vs `iterrows`
7.8×	threading on I/O-bound work — and **1.09×** on CPU-bound

The last pair is the one that answers most GIL questions in a single sentence.